

DocExtract — AI engineering reference

Engineering-level documentation of every `(src)` module and app: the functions inside, what they do, and *why* each module sits in the layer it does. Written for an engineer who will build, extend, or review the pipeline.

Companion to the mental model (intuition) and the technical reference (prose overview) — this one is the function-level detail.

How the package is layered, and the rule that governs it

shared/	foundations — imported by everyone, imports no one
custom/	the custom extraction pipeline (imperative, per-element)
search/	vector search + figure preprocessing
native/	the Databricks-native alternative track (declarative)

The one-way rule: `(shared/)` never imports from `(custom/)`, `(search/)`, or `(native/)`. `(native/)` never imports from `(custom/)`. This is what lets the two extraction tracks (custom and native) coexist behind one `(schema)` and one `(config)`. When you extend the system: foundations → `(shared/)`, pipeline steps → `(custom/)`, and never make `(shared/)` depend upward.

Why a module lives where it does — the test is *what it depends on and who depends on it*:

- If it defines a contract or setting everyone needs → **shared/**
- If it's a step in turning a document into claims → **custom/**
- If it's about retrieval or reading figures → **search/**
- If it's the platform-native declarative alternative → **native/**

shared/ — foundations

Everything here is imported by the pipeline; none of it imports the pipeline. That's the definition of "shared."

schema.py

Why shared: it defines the vocabulary every other module speaks — put it anywhere else and everything would depend upward into that layer. **Contents:**

- `(Modality)`, `(ChunkType)`, `(CitationTier)`, `(ReviewTier)` (enums) — the fixed vocabularies. `(CitationTier)` = *how* a value was obtained (provenance); `(ReviewTier)` = *where* it is in human review. Kept separate on purpose.
- `(Element)`, `(Claim)`, `(RegulatoryChunk)`, `(GoldMetric)` (pydantic models) — the four core data shapes that flow through the pipeline.
- `(content_hash(text))` — stable id for idempotent dedup.

- `parse_numeric(raw)` — tolerant number parsing (handles %, commas, parens).
- `build_chunk(raw, quarantine)`, `build_claim(raw, quarantine)` — construct a model from raw dict input, routing malformed input to quarantine instead of raising. *Flag, never reject, starts here.*

config.py

Why shared: the single control panel — every module reads settings from it. **Contents:**

- `Environment` + `detect_environment()` — local vs Databricks detection.
- `Team` (enum) — the canonical domains. Single source of truth for scoping.
- `ModelSpec`, `Thresholds`, `ClaimQuantPolicy`, `RoutingPolicy`, `StoragePaths`, `SearchConfig`, `AzureEndpoints` — grouped config blocks.
- `Thresholds.review_threshold(metric)` — the per-metric review bar (a misread ratio and a misread date don't share one threshold).
- `StoragePaths.slug()` / `layer_table()` / `layer_local()` — name the per-team_reporttype tables.
- `Config.model(key)`, `.is_databricks()`, the `*_api_key()` resolvers, `.validate()` — runtime accessors; secrets resolved here, never hard-coded.

llm_client.py

Why shared: the one gateway to the models — `extractor`, `llm_router`, and `claim_quant_matcher` all call through it, so it can't live in any one of them. **Contents:**

- `LLMResponse` — normalised response wrapper.
- `LLMClient.complete(model_key, prompt, content)` — text completion.
- `LLMClient.extract(model_key, prompt, content, ...)` — forced-JSON tool call (structured output, optionally multimodal for chart reads).

dictionary.py

Why shared: term canonicalisation is used by extraction *and* referenced by storage/reconciliation grouping. **Contents:**

- `canonical_metric(raw) -> (canonical, is_mapped)` — maps a metric name to its canonical key; the boolean flags whether it was recognised (unmapped terms flow through, they're just not cross-checked).
- `canonical_doctype(raw)` — same for document types.

metric_normaliser.py

Why shared: used by both `extractor` (tag each claim) and `validator` (reconcile), so it sits below both. **Contents:**

- `MeasureType`, `Period` (enums) — a value's structural shape.
- `Classification` + `classify_metric(name, value)` — infer shape from the name.
- `NormResult` + `check_compatible(a, b)` — are two claims like-for-like?

- `to_conflict_chunk(metric, a, b, ...)` — render an incompatibility as a searchable record.

routing.py

Why shared: the routing *policy* lives in config; this reads it. Used by `extractor`.

Contents:

- `RoutingDecision` — the chosen tier + reason + whether it was content-aware.
- `decide_start_tier(element, ...)` — pick the starting model tier from cheap signals *before* any API call.

prompts.py & prompt_registry.py

Why shared: prompts are consumed by extraction *and* the query router.

prompt_registry.py contents:

- `FewShot`, `PromptEntry` + `PromptEntry.render(body)` — a stored prompt.
- `register_prompt(entry, updated_by)` — self-serve write (latest wins).
- `get_prompt(team, report_type, kind)` — fetch, with default-seed fallback.
- `list_missing(pairs)` — which (team, report_type) have no registered prompt.

prompts.py contents:

- `build_prompt(team, report_type)` — assemble the extraction prompt, registry first then fallback.

eval_registry.py

Why shared: ground-truth answers are read by the evaluator (custom/) but are domain data, not pipeline logic — they belong with the other registries. **Contents:**

- `ExpectedValue` + `.matches(actual)` — one known-correct answer with a match rule (numeric-within-tolerance or exact) .
- `register_expected(entry, updated_by)` — self-serve write.
- `get_expected(...)`, `expected_for_document(...)` — fetch.
- `list_missing(pairs)` — which teams have registered no ground truth.

supervisor_session.py

Why shared: the chat session store is used by `llm_router` and surfaced by `monitoring`; it underpins the query side without being part of extraction. **Contents:**

- `Turn` — one Q&A exchange + optional feedback.
- `SupervisorSession.recent_turns()`, `.memory_context()`, `.has_history()` — conversational memory for reference resolution.
- `.record_turn(...)` — append + persist a turn (audit).
- `.add_feedback(turn_id, rating, note)` — attach thumbs up/down.
- `load_query_log()` — read the persisted Q&A log.

llm_router.py

Why shared: the query brain is the consumption-side entry point; it's not a step in document processing, so it doesn't belong in custom/. **Contents:**

- `Route` (enum) — SQL / narrative / both.
- `QueryRouter.classify(question)` — pick the route.
- `.generate_sql(question)` — NL→read-only SQL.
- `.resolve_references(question, session)` — rewrite a follow-up into a self-contained question using memory.
- `.answer(question, ..., session)` — the full flow: resolve → classify → SQL and/or retrieve → synthesise, with logging.
- `.run_sql(sql)` — execute a guarded SELECT.
- `is_safe_select(sql)` — the read-only guard.

audit_log.py, cost_tracker.py, gates.py, monitoring.py

Why shared: cross-cutting concerns used across the pipeline.

- **audit_log:** `Event`, `AuditLog.log()/trace()/to_rows()` — the traceability spine; `production` writes to it at every stage.
- **cost_tracker:** `CostTracker.record()/totals()`, `project(...)` — spend accounting + offline projection.
- **gates:** `Stage`, `SignOff` — governance checkpoints for release/deploy.
- **monitoring:** `RunMetrics`, `PipelineMonitor` (+ `NativePipelineMonitor`)
`.record()/load_runs()/summary()`; plus `supervisor_feedback_summary()` and `eval_accuracy_summary()` — aggregate dashboards over runs, feedback, and eval.

custom/ — the extraction pipeline

The imperative, per-element track. These are the stations a document passes through. They depend on `shared/` (and `search/` for figures) — never the reverse.

preprocessor.py

Why custom: it's step 1 of turning a file into claims — squarely pipeline. **Contents:**

- `preprocess_pdf / _docx / _xlsx / _pptx / _email(...)` — per-format parsers, each returning `Element`s. Text-layer content read locally (cheap); charts become FIGURE elements; text carries page + bbox; footnotes kept whole.
- `preprocess(path, ...)` — the dispatcher that picks a handler by extension.

extractor.py

Why custom: the core extraction step. Depends on 8 modules — the most connected node in the pipeline. **Contents:**

- `ExtractionResult` + `.track_cost(...)` — the output bundle + live cost.

- `Extractor.extract(elements, document_bytes, filename, page_image_provider)` — route → crop-zoom charts → forced-JSON call → escalate low-confidence → canonicalise + classify → tag provenance → quarantine malformed.

validator.py

Why custom: quality control on extracted claims — a pipeline step. **Contents:**

- `Conflict` — a flagged disagreement/impossibility.
- `reconcile(claims)` — group like-for-like, flag conflicts.
- `gate_for_review(claims, conflicts)` — split clean vs needs-review.

claim_quant_matcher.py

Why custom: an optional pipeline step linking narrative to metrics. **Contents:**

- `looks_like_claim(text)` — cheap prefilter.
- `ClaimQuantLink` — a resolved link.
- `match_claim(...)`, `match_all(...)` — retrieve-then-confirm linking.

ai_parse_client.py

Why custom: the OCR fallback used by the preprocessor when a page has no text layer — part of the parse step, so it sits in the pipeline layer (it reaches into `native/` only for the endpoint config). **Contents:**

- `ParsedBlock` — a block of OCR-extracted text.
- `AIParseClient.available()` — is the OCR endpoint configured?
- `AIParseClient.parse_page_image(image_bytes, ...)` — OCR a scanned page via the `databricks-ai-parse` endpoint; the preprocessor calls this only when the normal parse finds no text, and degrades to quarantine if it's unavailable.

reviewer.py

Why custom: applies a human decision to a claim — part of the review stage. **Contents:**

- `ReviewDecision` — an approve/override/reject.
- `review(...)` — construct a decision.
- `apply_decision(claim, decision)` — fold it in, updating provenance to HUMAN_* so the trail shows a human changed it.
- `decision_row(decision)` — render for persistence.

storage.py

Why custom: the medallion persistence step. **Contents:**

- `StorageManager` (base) `.append_jsonl()/ .read_jsonl()` — local backend.
- `DeltaLakeStorage` (Databricks) — `.build_gold_metric_rows()`, `.write_gold_metrics()/ .write_silver()/ .write_bronze()`,

`.write_quarantine()/.write_reviews()/.write_audit()` — per-domain, idempotent writes.

production.py

Why custom: the orchestrator that runs the custom pipeline end to end. **Contents:**

- `BatchResult` — the run tally (docs, gold, review, conflicts, cost, eval...).
- `process_batch(submissions, ..., link_claims, eval_every_n)` — the conductor: preprocess → extract → link → reconcile → gate → store → index → eval, with audit at every stage.

run_ingestion_job.py

Why custom: the batch entry point into the custom pipeline. **Contents:**

- `entity_ref_from_filename(name)` — parse entity id from filename.
- `discover_submissions(root)` — scan the volume for work.
- `main()` — CLI entry.

workflow_tasks.py

Why custom: the three Databricks Job tasks wrapping the pipeline. **Contents:**

- `ingest_task()`, `index_task()`, `healthcheck_task()` — the Jobs-graph tasks; index is its own re-runnable task, health-check fails red on abnormal runs.

evaluator.py

Why custom: it runs *during* the pipeline (sampled, per document), so it lives with the pipeline even though it reads the shared eval_registry. **Contents:**

- `is_sampled(doc_id, every_n)` — deterministic 1-in-N selection.
- `EvalRow`, `EvalResult` (+ `.accuracy`, `.override_rate`, `.summary()`).
- `evaluate_document(...)` — score claims vs registered + human-reviewed truth.
- `maybe_evaluate(...)` — pipeline entry; only the sampled fraction.
- `eval_summary()` — aggregate for the dashboard.

search/ — retrieval & figures

ai_search_store.py

Why search: it is the retrieval backend. Depends on shared, used by `llm_router` (query) and `production` (indexing). **Contents:**

- `AzureAISearchStore.ensure_index()` — provision HNSW + embeddings + semantic.
- `.index(chunk)` / `.index_many(chunks)` — push chunks.
- `.search(query, top_k, team_filter)` — hybrid, team-scoped retrieval.

figure_preprocessor.py

Why search: figure reading is a retrieval/vision concern, reused by the extractor but conceptually separate from it, so it's independently importable. **Contents:**

- `PageImageProvider`(ABC) + `PdfPlumberPageImageProvider.get_page_image(...)` — rasterise the source page.
 - `crop_and_zoom(page_image, bbox, padding)` — crop to the chart, upscale.
 - `prepare_figure_image(...)` — the end-to-end figure prep.
 - `encode_image_base64(image)`, `is_chart_like(element)` — helpers.
-

native/ — the Databricks-native alternative track

A *different way to produce the same Gold*, using the platform's declarative AI functions instead of the custom Python cascade. Independent of `custom/` — you can run either.

ai_functions_config.py

Why native: config specific to the native track. **Contents:** `AIFunctionsConfig`, `extract_fields_array()`, `classify_sql_template(...)` — the declarative extraction schema + SQL scaffold.

native_pipeline.py

Why native: the Lakeflow declarative pipeline itself. **Contents:** `bronze_sql()`, `silver_sql()`, `silver_exploded_sql()`, `gold_sql()` and the DLT table functions `bronze_documents / silver_claims / silver_claims_exploded / gold_metrics` — Bronze→Silver→Gold via AI functions.

vector_sync.py

Why native: bridges native-track Gold to the search index. **Contents:** `VectorSync.sync(delta_rows, ...)` — read newly-approved Gold, build chunks, push to `ai_search_store`.

apps/ — the human-facing UIs

Streamlit apps. Not `src` modules — they *import* the package and present it. Each has a `run()` entry and is launched with `streamlit run`.

analyst_app.py

What it does: inspect one document's extraction *live* — upload → preprocess → extract → view each claim's value, confidence, provenance, and what's flagged. Front of the pipeline; reads nothing from storage. *Depends on: extractor, preprocessor, config.*

supervisor_app.py

What it does: the front desk — a supervisor asks questions and gets grounded, cited answers, with conversational memory and thumbs up/down feedback. Consumption end. *Depends on: llm_router, supervisor_session, ai_search_store, config.*

monitoring_app.py

What it does: the ops dashboard — aggregate health across all runs (throughput, cost, review backlog, error rate), plus feedback and eval summaries. *Depends on: monitoring (which pulls from evaluator + supervisor_session).*

Populating the three registries — the recommendation

`(prompt_registry)`, `(eval_registry)`, and `(dictionary)` all hold **domain knowledge that teams supply**, not code. The question is *how* teams get their content in. Here's the honest trade-off and a clear recommendation.

The two options

A. Teams write to the registry directly (call `(register_prompt)` / `(register_expected)`, or edit the dictionary mappings).

- *Pros:* single source of truth; no extraction step; changes are live immediately; every write is already audited (`(updated_by)`).
- *Cons:* teams need to touch code/notebooks or a small tool; non-technical teams may find it a barrier.

B. Teams fill an Excel sheet; you extract it into the registry.

- *Pros:* teams work in a familiar tool; you review before it lands; easy to hand a template to a non-technical team.
- *Cons:* an extra extraction step you own and must keep running; a *second* copy of the truth (the sheet vs the registry) that can drift; no live audit of who changed what until you ingest.

The recommendation

Use Excel-as-intake for the *initial* population, and the registry as the *system of record* thereafter — with a one-way ingest, not a two-way sync.

Concretely:

- Give each team a **template sheet** per registry (columns matching the registry fields — for eval: `(team, report_type, entity_ref, metric, expected_value, match, tolerance)`; for prompts: `(team, report_type, kind, example_input, example_output)`; for dictionary: `(raw_term, canonical)`).
- Write a small **importer** that reads the sheet and calls `(register_expected)` / `(register_prompt)` / appends dictionary entries — so the sheet is *input*, the registry is *truth*. The registry, not the sheet, is what the pipeline reads.
- After initial load, **prefer direct registry writes** for ongoing changes (they're audited and live), and treat the sheet as a re-usable intake form for bulk additions, not a maintained parallel copy.

Why this way, honestly: the failure mode to avoid is the sheet becoming a permanent second source of truth that silently diverges from the registry — exactly the drift

problem we've been careful about elsewhere. Excel is a great *doorway* (familiar, reviewable, good for a non-technical team's first bulk entry); it's a poor *system of record* (no audit, easy to fork, manual re-ingest). So use it as the doorway, make the registry the record, and keep the ingest one-way.

One caveat on the dictionary specifically: unlike the other two, the dictionary maps synonyms → canonical names and lives partly *in code* today (`(METRIC_CANONICAL)` / `(DOCTYPE_CANONICAL)` dicts). If teams will maintain it regularly, it's worth promoting it to a registry like the other two (data, not code) so the same Excel-doorway pattern applies. If it changes rarely, leaving it in code and editing it directly is fine. That's a judgment call on how often your domains' vocabularies shift.

The rule of thumb

Excel to get it *in* (familiar, reviewable, bulk). Registry to *keep* it (audited, live, single source). One-way ingest between them — never a two-way sync, or you'll have two truths.